

Welcome back

How was lab? Was anything confusing?

Lecture notes: <https://tinyurl.com/4ypzsmzu>

Review

Last time we covered functions. Functions are critical to programming.

Syntax for functions

The general syntax for functions is:

```
def function_name(<arguments>):  
    <body>  
    return <expression>
```

There are several things to note about the syntax (we will go through examples):

- You can have zero, one, or more arguments.
- You need the `:` after the function
- The body can be complex!
- You technically don't need a return statement at the end, but we will almost always have a return at the end

```
def minimum_two(x, y):  
    # x = 1  
    # y = 2  
    if x < y:  
        return x  
    else:  
        return y  
  
minimum_two(1, 2)  
  
def min_list(l):  
    minimum = l[0]  
    for x in l[1:]:  
        minimum = minimum_two(minimum, x)  
    return minimum
```

```
min_list([6, 5, 4, 3, 2, 1])

def min_three(x, y, z):
    min_xy = minimum_two(x, y)
    min_xyz = minimum_two(min_xy, z)
    return min_xyz

min_three(1, 2, 3)

min(1, 2)

max(1, 2)
```

You cannot use certain words in Python as variables

Conversion:

- `int`
- `float`
- `bool`
- `str`
- `list`
- `dict`

Build in:

- `min`
- `max`
- `sum`

Reserved keywords:

- `if`
- `else`
- `for`
- `while`

DO NOT USE THESE AS VARIABLE NAMES

Function examples

We will go through examples of functions

```

# This function takes in `lst` as an argument
# and returns the sum of all of the elements of lst
def sum_list(lst):
    # We will use a variable s to keep track of the sum so far
    # And we will use a for loop to go through all of the elements of
    lst
    # And add to s

    # s = 0
    # for x in lst:
    #     s += x
    # return s

    # ... using a while loop to go thorough all of the elements of lst
    # lst = [4, 2, 8, 1]
    s = 0
    index = 0
    while index < len(lst):
        s += lst[index]
        index += 1
    return s

sum_list([4, 2, 8, 1])

# What the function does, independent of _how_ it is done
def function(args):
    # How to implement the function
    body
    return ...

word = "word"
reversed_word = "drow"

# word = "racecar"
# reversed_word = "racecar"

word == reversed_word

# is_palindrome returns True if word is a palindrome
# It returns false otherwise
def is_palindrome(word):
    # We will first create reversed_word, which is word reversed
    # To do this, we will go through word in reverse order using a for
    loop
    # And create a new string reversed_word, with the characters in
    reversed order
    # After we have reversed_word, we will compare word and
    reversed_word
    reversed_word = ""
    for index in range(len(word) - 1, -1, -1):

```

```

        reversed_word += word[index]
    return word == reversed_word

is_palindrome("racecar")

word = "word"

for x in range(len(word) - 1, -1, -1):
    print(x)

def sum_list(l):
    s = 0
    for x in l:
        s += x
    return s

sum_list([1, 2, 3, 4, 5])

def is_palindrome(s):
    for ind in range(len(s)):
        if s[ind] != s[len(s) - ind - 1]:
            return False
    return True

print(is_palindrome("abccba"))
print(is_palindrome("abcba"))
print(is_palindrome("abcbab"))

```

Functions can call other functions

```

# This function adds two numbers together and returns the result
def add_two_numbers(x, y):
    # We will take x and y and return x + y
    return x + y

add_two_numbers(1, 2)

# This function adds three numbers together and returns the result
def add_three_numbers(x, y, z):
    # ...
    sum_two = add_two_numbers(x, y)
    sum_three = add_two_numbers(sum_two, z)
    return sum_three

add_three_numbers(1, 2, 3)

```

Functions can *call themselves*

```
def calls_itself(x):
    # x = -1
    if x < 0:
        return 0
    else:
        print(x)
        return calls_itself(x - 1) # What is the result when x = 0?

calls_itself(0)

def sum_list(l):
    # l = []
    if len(l) == 0:
        return 0
    else:
        #         1 + sum_list([])
        #         1 + 0
        return l[0] + sum_list(l[1:])

sum_list([1, 2, 3])

# l = [1, 2, 3]
# l[0] = 1
# l[1:] = [2, 3]
# 1 + sum_list([2, 3]) = 1 + 5 = 6

# l = [2, 3]
# l[0] = 2
# l[1:] = [3]
# 2 + sum_list([3]) = 2 + 3 = 5

# l = [3]
# l[0] = 3
# l[1:] = []
# 3 + sum_list([]) = 3 + 0 = 3

# l = []

1

l = [1]
l[1:]

[]
```

WARNING: IF A FUNCTION CALLS ITSELF, IT MUST STOP AT SOME POINT

```
# WARNING
# WARNING
# WARNING
# DO NOT RUN THIS CELL

def calls_itself(x):
    print(x)
    return calls_itself(x - 1)

calls_itself(2)
```

Recursion

When a function calls itself, it is called *recursion*. This is a powerful, yet confusing concept.

```
x = 2
nth = 5

2 ** 0

1

def nth_power(x, nth):
    # x = 2
    # nth = 3
    if nth == 0:
        return 1
    # 2 * nth_power(2, 2)
    # 2 * 4 = 8
    return x * nth_power(x, nth - 1)

# nth_power(2, 0) = 1
# nth_power(2, 1) = 2
# nth_power(2, 2) = 4
# nth_power(2, 3) = 8

4

# F_0 = 0
# F_1 = 1
# F_n = F_{n-1} + F_{n-2}

def fibonacci(n):
    # n = 3
```

```

    if n < 0:
        return 0
    if n == 0 or n == 1:
        return n
    #     fibonacci(2)      + fibonacci(1)
    #     1                + 1
    return fibonacci(n - 1) + fibonacci(n - 2)

# fibonacci(0) = 0
# fibonacci(1) = 1
# fibonacci(2) = 1
fibonacci(3)

2

def factorial(x):
    if x == 1:
        return 1
    return x * factorial(x - 1)

factorial(5)

def sum_n(n):
    if n == 1:
        return 1
    return n + sum_n(n - 1)

sum_n(5)

def reverse(s):
    if len(s) == 0:
        return s
    else:
        return reverse(s[1:]) + s[0]

reverse("hi")

def is_palindrome(s):
    if len(s) <= 1:
        return True
    return (s[0] == s[-1]) and is_palindrome(s[1:-1])

print(is_palindrome("abccba"))
print(is_palindrome("abcba"))
print(is_palindrome("abcbab"))

for x in range(3, -1, -1):
    print(x)

```

3
2
1
0